Data Wrangling — Scan It, Store It, Walk It

Class 10 — In-Class Lab Sheet (Getting input into the right structure — lists, matrices, 3-D arrays, dictionaries — using `while` only.)

Two muscles solve every coding problem: **(1)** get the data into the right container, and **(2)** think

about what to do with it. Today is pure muscle **(1)** — the problems are logically easy on purpose, so

all your effort goes to the *plumbing*: scanning input, choosing a structure, filling it, and walking it.

Everything uses `while` loops and indexing — no `for`, no shortcuts.

Layout: Section A — trace scanning code (predict the output). Section B — coding problems on **Codez**, graded easy → hard, each with handwritten notes. Section C — calibration.

Section A — Trace the Scan (predict, then run)

A1

Input fed to the program: `4` then `7 7 2 9` (one per line).

```
n = int(input())
arr = []
i = 0
while i < n:
    arr.append(int(input()))
    i += 1
print(len(arr), arr[0], arr[n-1])
```

Predicted: _____ Actual: _____

A2

```
m = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
print(m[1])
print(m[1][2])
print(len(m), len(m[0]))
```

Predicted (3 lines): _____ _____ _____

A3

For the same `m`, what does this print?

```
j = 0
while j < 3:
    i = 0
    while i < 3:
        print(m[i][j], end=" ")
        i += 1
    j += 1
```

Predicted: _____ *(row-wise or column-wise?)* _____

Section B — Coding Problems (code on Codez)

Read from stdin, print to stdout. Submit on the judge, then fill the handwritten notes below each.

B1 — Sum of N Numbers `[Easy]` `[LMS: dw-sum-n]`

First line: `n`. Next `n` lines: one integer each. Print their sum.

```
Input:  5 / 10 / 20 / 30 / 40 / 50          Output:  150
```

```
First attempt — if it failed, which input broke it and what was the flaw?
_____________________________________________________________________
Proof — why is your final solution always correct?
_____________________________________________________________________
Comfort check (list): create a list, append to it, read arr[i]?   Y / N
Reflection — what you learned; a mistake repeated before & how to avoid it:
_____________________________________________________________________
```

B2 — Maximum and Minimum `[Easy]` `[LMS: dw-maxmin]`

One line of space-separated integers. Print the largest and the smallest (do it by scanning — don't rely on built-ins).

```
Input:  3 8 1 9 4          Output:  9 1
```

```
First attempt — if it failed, which input broke it and what was the flaw?
_____________________________________________________________________
Proof — why is your final solution always correct?
_____________________________________________________________________
Comfort check (split scan): split a line and convert each piece to int?  Y / N
Reflection — what you learned; a mistake repeated before & how to avoid it:
_____________________________________________________________________
```

B3 — Count Above Average `[Medium]` `[LMS: dw-above-avg]`

First line `n`, then `n` integers (one line, space-separated). Print how many of them are strictly greater than the average of all `n`.

```
Input:  5 / 3 8 1 9 4          Output:  2        (average is 5.0; 8 and 9 beat it)
```

```
First attempt — if it failed, which input broke it and what was the flaw?
_____________________________________________________________________
Proof — why is your final solution always correct?
_____________________________________________________________________
Comfort check (two passes): scan once to total, again to compare?   Y / N
Reflection — what you learned; a mistake repeated before & how to avoid it:
_____________________________________________________________________
```

B4 — Matrix Total `[Easy]` `[LMS: dw-matrix-sum]`

First line: `R C`. Next `R` lines: `C` integers each. Print the sum of all cells.

```
Input:  3 3 / 1 2 3 / 4 5 6 / 7 8 9          Output:  45
```

```
First attempt — if it failed, which input broke it and what was the flaw?
_____________________________________________________________________
Proof — why is your final solution always correct?
_____________________________________________________________________
Comfort check (2D list): build a list of lists, read m[i][j]?   Y / N
Reflection — what you learned; a mistake repeated before & how to avoid it:
_____________________________________________________________________
```

B5 — Row Sums and Column Sums `[Medium]` `[LMS: dw-row-col-sums]`

Read an `R × C` matrix. On line 1 print the `R` row-sums (space-separated); on line 2 print the `C` column-sums.

```
Input:  3 3 / 1 2 3 / 4 5 6 / 7 8 9
Output: 6 15 24
        12 15 18
```

```
First attempt — if it failed, which input broke it and what was the flaw?
_____________________________________________________________________
Proof — why is your final solution always correct?
_____________________________________________________________________
Comfort check (column walk): traverse a matrix column-by-column?   Y / N
Reflection — what you learned; a mistake repeated before & how to avoid it:
_____________________________________________________________________
```

B6 — Diagonal Sums `[Medium]` `[LMS: dw-diag-sums]`

Read a square `N × N` matrix. Print the sum of the main diagonal and the sum of the anti-diagonal.

```
Input:  3 / 1 2 3 / 4 5 6 / 7 8 9          Output:  15 15
```

```
First attempt — if it failed, which input broke it and what was the flaw?
_____________________________________________________________________
Proof — why is your final solution always correct?
_____________________________________________________________________
Comfort check (index math): express anti-diagonal cell as m[i][N-1-i]?  Y / N
Reflection — what you learned; a mistake repeated before & how to avoid it:
_____________________________________________________________________
```

B7 — Transpose `[Medium]` `[LMS: dw-transpose]`

Read an `R × C` matrix and print its transpose (columns become rows), one row per line.

```
Input:  3 3 / 1 2 3 / 4 5 6 / 7 8 9
Output: 1 4 7
        2 5 8
        3 6 9
```

```
First attempt — if it failed, which input broke it and what was the flaw?
_____________________________________________________________________
Proof — why is your final solution always correct?
_____________________________________________________________________
Comfort check (2D build): build a NEW list of lists from an old one?  Y / N
Reflection — what you learned; a mistake repeated before & how to avoid it:
_____________________________________________________________________
```

B8 — Diagonal Traversal `[Hard]` `[LMS: dw-diag-traverse]`

Read an `R × C` matrix. Print its cells grouped by anti-diagonal (cells where `i + j` is equal), sweeping `i + j` from 0 upward — all on one line.

```
Input:  3 3 / 1 2 3 / 4 5 6 / 7 8 9          Output:  1 2 4 3 5 7 6 8 9
```

```
First attempt — if it failed, which input broke it and what was the flaw?
_____________________________________________________________________
Proof — why is your final solution always correct?
_____________________________________________________________________
Comfort check (derived index): given d and i, compute j = d - i and bounds-check?  Y / N
Reflection — what you learned; a mistake repeated before & how to avoid it:
_____________________________________________________________________
```

B9 — Diagonal Zigzag Traversal `[Hard]` `[LMS: dw-diag-zigzag]`

Same anti-diagonals as B8, but reverse the walking direction on alternate diagonals so the path snakes.

```
Input:  3 3 / 1 2 3 / 4 5 6 / 7 8 9          Output:  1 2 4 7 5 3 6 8 9
```

```
First attempt — if it failed, which input broke it and what was the flaw?
_____________________________________________________________________
Proof — why is your final solution always correct?
_____________________________________________________________________
Comfort check (direction flag): switch loop direction based on d % 2?  Y / N
Reflection — what you learned; a mistake repeated before & how to avoid it:
_____________________________________________________________________
```

B10 — Spiral Print `[Hard +]` `[LMS: dw-spiral]`

Read an `R × C` matrix. Print its cells in spiral order: top row →, right column ↓, bottom row ←, left column ↑, shrinking inward — all on one line.

```
Input:  3 3 / 1 2 3 / 4 5 6 / 7 8 9          Output:  1 2 3 6 9 8 7 4 5
```

```
First attempt — if it failed, which input broke it and what was the flaw?
_____________________________________________________________________
Proof — why is your final solution always correct?
_____________________________________________________________________
Comfort check (boundaries): manage top/bottom/left/right shrinking inward?  Y / N
Reflection — what you learned; a mistake repeated before & how to avoid it:
_____________________________________________________________________
```

B11 — Cube Sum (3-D array) `[Medium]` `[LMS: dw-cube-sum]`

First line: `A B C`. Then `A` blocks, each a `B × C` matrix (blocks separated as plain rows). Build a 3-D array and print the sum of every element.

```
Input:  2 2 2 / 1 2 / 3 4 / 5 6 / 7 8          Output:  36
```

```
First attempt — if it failed, which input broke it and what was the flaw?
_____________________________________________________________________
Proof — why is your final solution always correct?
_____________________________________________________________________
Comfort check (3D list): reach cube[a][b][c] with three nested whiles?  Y / N
Reflection — what you learned; a mistake repeated before & how to avoid it:
_____________________________________________________________________
```

B12 — Scoreboard Lookup (dictionary) `[Medium]` `[LMS: dw-dict-lookup]`

First line `n`, then `n` lines of `name score`. Then a line `q`, then `q` query names. For each query print `name score`. Also, on the last line, print the name and score of the top scorer.

```
Input:  3 / alice 90 / bob 85 / cara 93 / 2 / bob / cara
Output: bob 85
        cara 93
        cara 93
```

```
First attempt — if it failed, which input broke it and what was the flaw?
_____________________________________________________________________
Proof — why is your final solution always correct?
_____________________________________________________________________
Comfort check (dict): create a dict, store d[key]=value, look one up?  Y / N
Reflection — what you learned; a mistake repeated before & how to avoid it:
_____________________________________________________________________
```

Section C — Calibration (60 seconds, honest)

(a) Which structure still feels least comfortable to *declare and fill from scratch*? Circle one:

```
1-D list    |    2-D matrix (list of lists)    |    3-D array    |    dictionary
```

(b) Which matrix traversal fought back hardest — column-wise, diagonal, zigzag, or spiral? And was the trouble the *index math* or the *loop structure*?

```
_____________________________________________________________________
```

(c) One sentence: which scanning pattern will you drill before the next class?

```
_____________________________________________________________________
```